НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Комп'ютерний практикум № 2

з дисципліни «**Оброблення надвеликих масивів даних**»

Тема: «Класифікація та кластеризація інформації»

Виконав:
студент групи ІП-з41мп
Дишкант Лариса Леонідівна

Перевірив:
к.т.н. доцент кафедри ІПІ
Олійник Ю.О.

Київ 2024

Завдання

Необхідно методами машинного навчання із застосування мов програмування Phyton, Java, Scala, R та технології Apache Spark провести класифікацію та кластеризацію числової чи категорійної інформації. Для цього виконати наступні дії:

а) знайти набір даних для класифікації чи кластеризації даних в структурованому вигляді. Неменше 100 тис. рядків (бажано 1 млн.);

б) обрати не менше 2х методів для класифікації та 2 методи кластеризації даних. Обґрунтувати свій свій вибір.

в) використати крос-валідацію з 5 фолдами для однієї моделі класифікації.

г) виконати підбір гіперпараметрів для однієї моделі класифікації. Вивести ROC-криву, матрицю неточностей, міри точності для навчених моделей, Провести аналіз результатів підбору та оцінити результати експериментів.

д) виконати підбір параметрів для виконання кластеризації. Провести аналіз результатів підбору та оцінити результати експериментів.

Для виконання завдання використаємо датасет - Титанік, але для роботи додамо ще даних, згенерувавши їх за допомогою коду. Збільшимо наш датасет до потрібно відповідно вимог (рисунок 1). Результат можем побачити на рисунку 2.

```python
# Завантаження датасету Титанік з Kaggle (менший набор)
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"

# Читання CSV-файлу
original_data = pd.read_csv(url)

print(f"Initial rows: {original_data.shape[0]}")

# Функція для випадкового змішування даних
def create_variations(data, num_variants):
    variations = []
    for _ in range(num_variants):
        # випадкове вибірка з деякими варіаціями
        temp_data = data.copy()
        temp_data['Age'] = temp_data['Age'] + np.random.randn(data.shape[0])  # Додаємо невелику випадкову зміну до віку
        temp_data['Fare'] = temp_data['Fare'] + np.random.randn(data.shape[0])  # Додаємо випадкову зміну до платні
        variations.append(temp_data)
    return pd.concat(variations, ignore_index=True)

# Кількість повторень для досягнення 100,000 рядків
required_rows = 100000
num_variants = required_rows // original_data.shape[0] + 1  # не менше, ніж потрібно в один раз

# Генерація варіантів
extended_data = create_variations(original_data, num_variants)

# Вибір тільки перших 100,000 рядків
final_data = extended_data.head(required_rows)
```

Рисунок 1 - Код для отримання потрібного датасета

```
Initial rows: 891
Final rows: 100000
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex        Age  \
0                            Braund, Mr. Owen Harris    male  21.690994
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  37.947212
2                             Heikkinen, Miss. Laina  female  26.832902
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  33.627539
4                           Allen, Mr. William Henry    male  33.875024

   SibSp  Parch            Ticket       Fare Cabin Embarked
0      1      0         A/5 21171   7.467659   NaN        S
1      1      0          PC 17599  70.365722   C85        C
2      0      0  STON/O2. 3101282   8.761421   NaN        S
3      1      0            113803  52.107021  C123        S
4      0      0            373450   8.090336   NaN        S
```

Рисунок 2 - Результат створення датасету

Обрати не менше 2х методів для класифікації та 2 методи кластеризації даних. Обґрунтувати свій свій вибір.

Для виконання завдання було обрано Логістичну регресію та Дерева рішень. Зупинилася на Логістичній регресії тому що вона є одна з найпростіших та зрозумілих моделей для класифікації, яка дає чіткі інтерпретації коефіцієнтів та дозволяє зрозуміти який вплив мають ознаки на виживання. Також вона досить швидка в начанні та прогнозуванні, а це для великих датасетів важливо. Підходить для біноміальних результатів та у відсутності складних взаємозв'язків досить ефективна. Другою розглядалася модель дерева рішень, тому що легко обробляє числові та категоріальні змінні, що є в даному датасеті. Також досить добре працюють з відсутніми даними і можуть адаптуватися до пропусків. Обрання цих двох моделей є раціональним, оскільки дані методи відповідають природі даних, прості у впровадженні та надають зрозумілі результати.

Перед навчання моделей було дані оброблено: заповнено пропуски, перетворення полів у числові значення, трансформація міток зроблена (рисунок 3).

```
# Обробка даних: заповнення відсутніх значень
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, avg


# Заповнення відсутніх значень
mean_age = titanic_data.select("Age").agg({"Age": "avg"}).first()[0]
titanic_data = titanic_data.fillna({"Age": mean_age, "Embarked": "S"})  # Заповнюємо відсутні значення

# Видалення непотрібних колонок
titanic_data = titanic_data.drop("Ticket", "Cabin", "Name")

# Перетворення полів у числові значення
titanic_data = titanic_data.withColumn("Sex",
    when(col("Sex") == "male", 1).otherwise(0).cast("double"))  # Заміна 'male' і 'female' на 1 та 0
titanic_data = titanic_data.withColumn("Embarked",
    when(col("Embarked") == "S", 1).when(col("Embarked") == "C", 2).when(col("Embarked") == "Q", 3).otherwise(0

# Трансформація міток для класифікації
titanic_data = titanic_data.withColumnRenamed("Survived", "label")
```

Рисунок 3 - Обробка даних

Використати крос-валідацію з 5 фолдами для однієї моделі класифікації. Крос Валідація використали на логістичній регресії (рисунок 4).

```
[25] # 1. Логістична регресія з крос-валідацією

     lr = LogisticRegression(featuresCol='features', labelCol='label')
     paramGrid = ParamGridBuilder().addGrid(lr.regParam, [0.1, 0.01]).build()

     crossval = CrossValidator(estimator=lr, estimatorParamMaps=paramGrid, evaluator=BinaryClassificationEvaluator(), numFolds=5)
     cvModel = crossval.fit(train_data_transformed)
```

Рисунок 4 - Використання крос валідації на Логістичній регресії

Виконати підбір гіперпараметрів для однієї моделі класифікації. Вивести ROC-криву, матрицю неточностей, міри точності для навчених моделей, Провести аналіз результатів підбору та оцінити результати експериментів.

Підбір гіперпараметра зробили по грідсітці для Логістичної регресії (рисунок 4) **ROC-криву** побудували для обох моделей (рисунок 5 та рисунок 6)

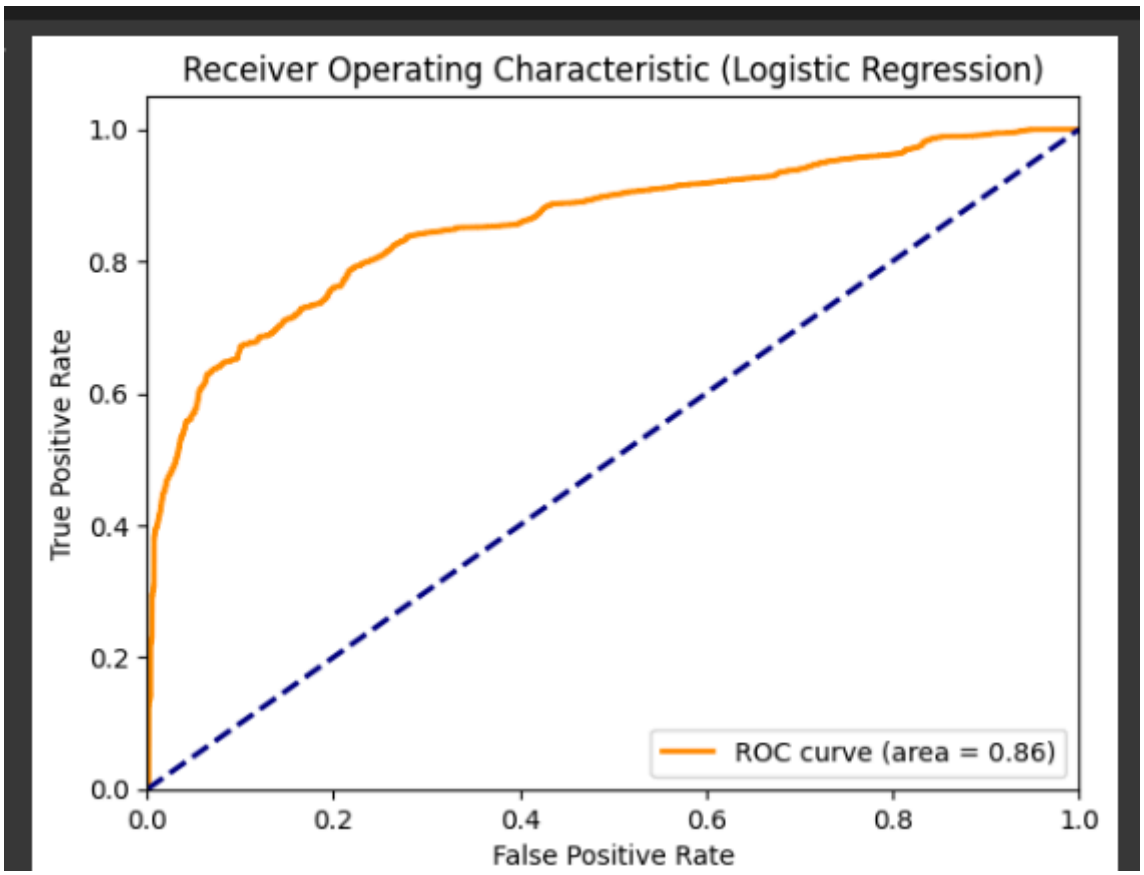

Рисунок 5 - ROC-крива для Логістичної регресії

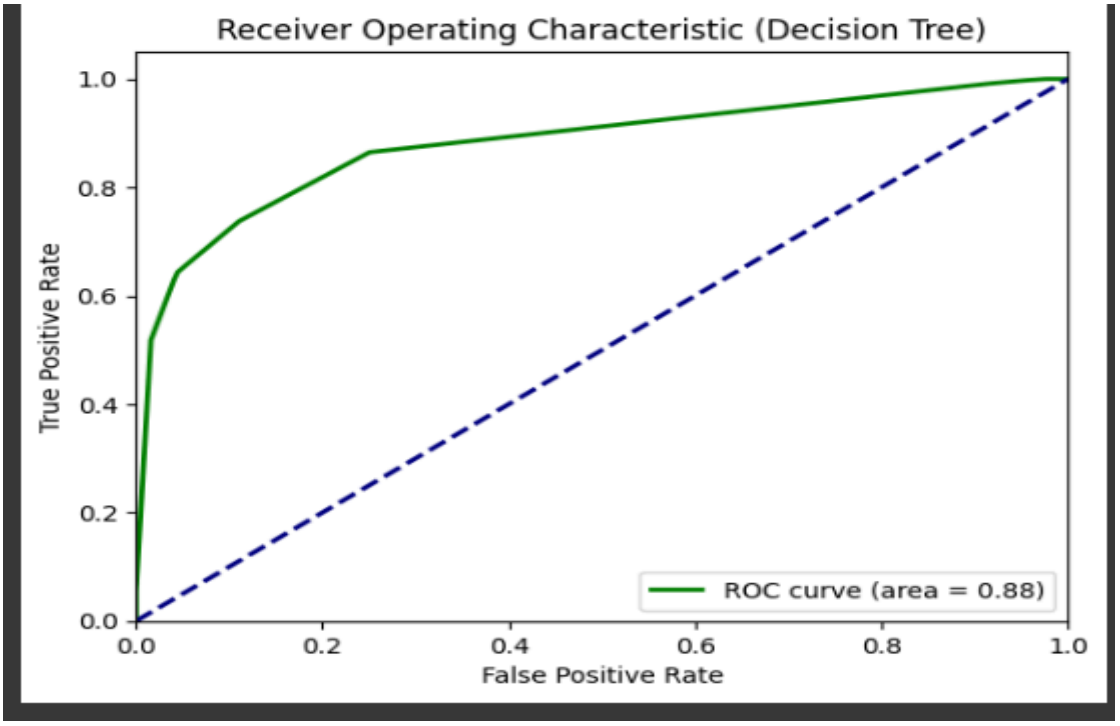

Рисунок 6 - ROC-крива для Дерев рішення

Як бачимо з графіків результат практично однаковий, досить непоганий, але можна покращити спробою використати нейромережу та відповідно підібрати гіперпараметри. На тестових даних моделі показали різні результати та відповідно гірші ніж на навчальній вибірці, але це нормально. А от сам результат показує, що дерева краще справляються з завданням, але 83% це не достатньо гарний результат, щоб вважати що модель можна залишити на такому рівні (рисунок 7)

```
Accuracies:
Logistic Regression Accuracy: 0.7991
Decision Tree Accuracy: 0.8368
```

Рисунок 7 - Результат двох моделей на тестових даних

```
Decision Tree Metrics:
Precision: 0.8987
Recall: 0.6436
F1 Score: 0.7500
```

Рисунок 8 - Метрики

```
Confusion Matrix for Logistic Regression:
Predicted    0.0   1.0
Actual
0          10715  1698
1           2326  5294

Confusion Matrix for Decision Tree:
Predicted    0.0   1.0
Actual
0          11860   553
1           2716  4904
```

Рисунок 9 - Матриці неточності для обох моделей

Матриця це дуже корисний інструмент для оцінки точності алгоритму класифікації. Він дає зрозуміти, як конструюються помилки під час класифікації. Тобто розглядаючи нашу матрицю бачимо, що Логістична регресія правильно класифікувала позитивні випадки у 5294, правильно

класифікувала негативні випадки у 10715. Але помилково класифікувала як позитивні у 1698 та помилково класифікувала як негативні у 2326. У дерев рішення результати кращі: правильно класифіковані позитивні випадки - 4904, правильно класифіковано негативні випадки - 11860, а помилково класифіковані як позитивні - 553 та 2716 як негативні. З огляду на це і якщо обирати тільки між даними моделями на наний момент то кращою буде модель дерева рішень.

Кластеризація

Для даного завдання було обрано KMeans рисунок 10

Кластеризація: KMeans та Bisecting KMeans

```
[40] # 1. KMeans
     kmeans = KMeans(featuresCol='features', k=2)
     kmeans_model = kmeans.fit(train_data_transformed)

     # Оцінка KMeans
     kmeans_predictions = kmeans_model.transform(train_data_transformed)
     centers = kmeans_model.clusterCenters()

[41] # 2. Bisecting KMeans
     bisecting_kmeans = BisectingKMeans(featuresCol='features', k=2)
     bisecting_kmeans_model = bisecting_kmeans.fit(train_data_transformed)

     # Оцінка Bisecting KMeans
     bisecting_predictions = bisecting_kmeans_model.transform(train_data_transformed)

[42] # Виведення деяких результатів
     print("KMeans Centers:")
     for center in centers:
         print(center)

     KMeans Centers:
     [ 2.37382662  0.66365582 29.63577936  0.51240006  0.35056256 23.51058882
       1.36021216]
     [  1.          0.33070037  31.48003123   0.67298578   0.9705108
      206.85000811   1.42943655]
```

Рисунок 10 - Кластеризація за допомогою двох обраних моделей

Для другого варіанта обрано варіація алгоритму KMeans, але яка використовується для розподілу даних на кілька кластерів, прагнучи бути більш ефективною та інтерпретованою в порівнянні із стандартною версією. Основна ідея в розділені одного кластера на два і так повторювати процес доки не буде досягнуто бажаної кількості кластерів. Його використовують щоб уникнути проблеми з локальними мінімумами. Але при своїх плюсах він має мінуси - час виконання.

Кластеризація для датасету Титанік надає можливість дослідити залежності між різними ознаками, тобто дослідити дані на більш глибокому рівні та виявити патерни, які можуть залишитися непоміченими за допомогою традиційних методів аналізу. Тобто дасть розуміння причини виживання чи загибелі на борту “Титаніка”.